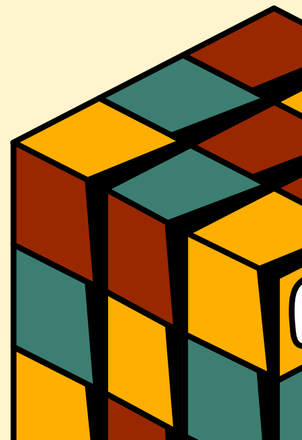
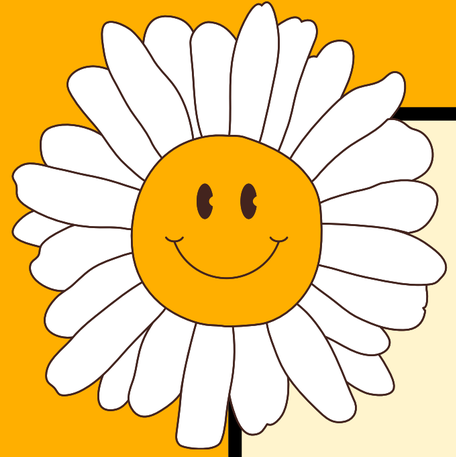
FYP Part 1

APPLYING MACHINE LEARNING TECHNIQUES TO PREDICT SOFTWARE DEFECTS

Name: Lew Eng Jean
Student ID: 2414UC2414K
Project ID: FYP01-SE-T2430-0144
Supervisor: Ms. Venushini A/P Rajendran
Moderator: Dr. Ng Sew Lai

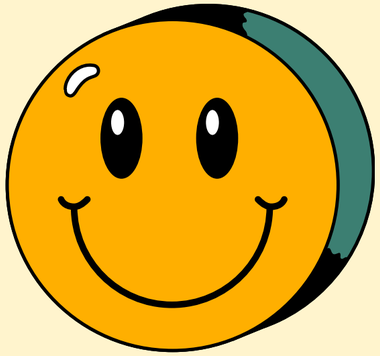
10 February, 2025





Introduction

Amidst the fast-paced advancements in technology today, the Software Defects Prediction (SDP) utilizing machine learning (ML) techniques is essential for improving software quality and ensuring timely delivery. The target of this project is to address the challenges about class imbalance in datasets and the high computational complexity in SDP.

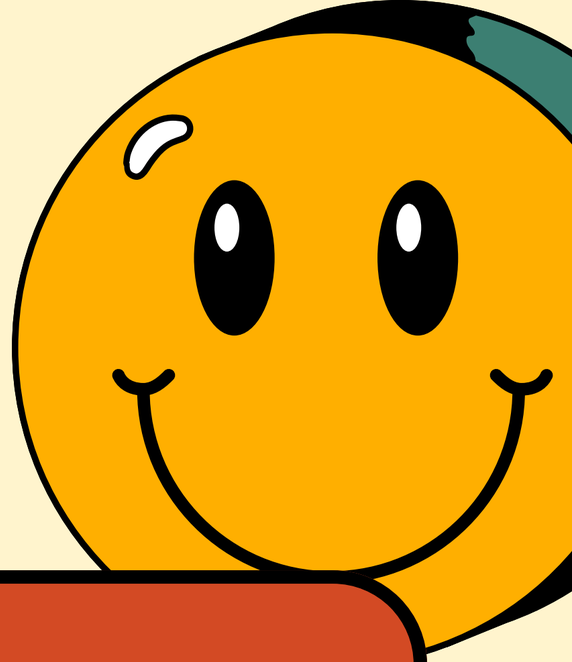


Introduction

1.1 Background of the study

- Software Development Complexity:
  - - Involves multiple stages: requirement gathering, deployment, maintenance.
  - - Early detection of defects is crucial to avoid financial losses, security issues, operational failures.
- Limitations of Traditional Testing:
  - Time-consuming
  - Resource-intensive
- Automated Defect Prediction Models:
  - Attractive alternative to traditional testing.
  - Uses ML to analyze historical defect data.
  - Identifies patterns indicative of future defects.

Problem Statement



Class Imbalance in Datasets

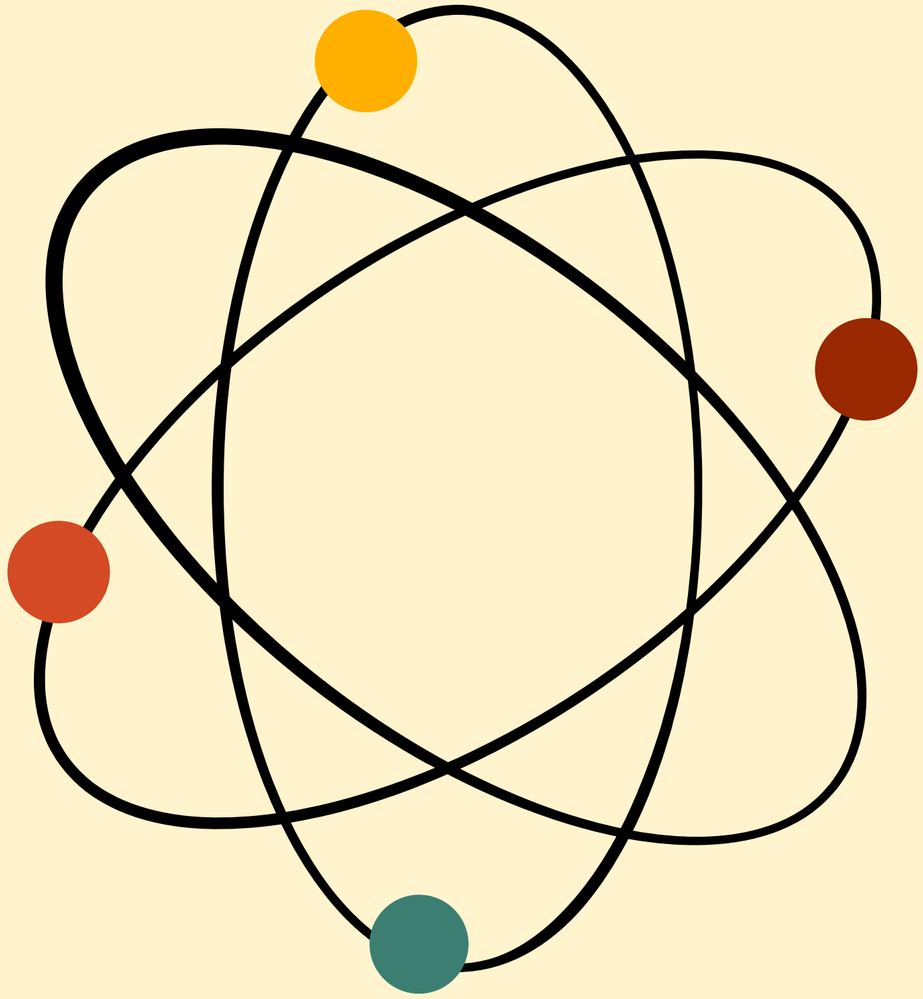
- Many datasets used in SDP contain disproportionately more non-defective than defective instances, leading to biased predictions and unreliable models.

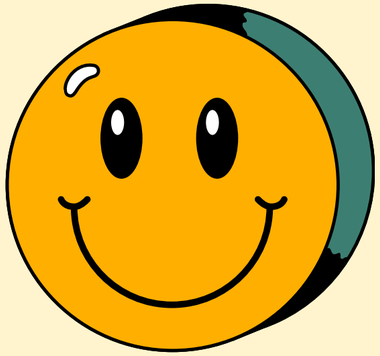
High Computational Complexity

- The large feature sets in datasets increase computational overhead, reducing the efficiency and scalability of ML models.

Research Purpose

- To enhance SDP by addressing two major problems
- Aims to develop a hybrid ML model that leverages RF and SVM to achieve better accuracy and efficiency
- Contributes to improving software quality, reducing maintenance costs, facilitating more reliable defect detection processes in swe, and bring awareness to the effectiveness of hybrid approaches





Objectives

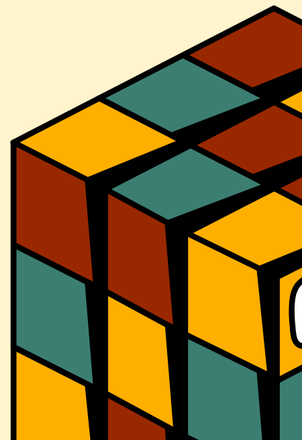
- To develop a hybrid ML model that addresses class imbalance and improves prediction accuracy.
- To reduce computational complexity and time consumption in SDP.
- To test the accuracy of the hybrid ML model.

Research Questions

HOW DOES A HYBRID ML MODEL EFFECTIVELY ADDRESS CLASS IMBALANCE IN SDP DATASETS?

WHAT TECHNIQUES CAN BE EMPLOYED TO REDUCE COMPUTATIONAL COMPLEXITY AND TIME CONSUMPTION IN SDP?

HOW ACCURATE IS THE HYBRID ML MODEL IN PREDICTING SOFTWARE DEFECTS COMPARED TO TRADITIONAL METHODS?



Project Scope

Class Imbalance in Datasets

- Develop a hybrid ML model to address class imbalance and improve prediction accuracy.
- Analyze existing SDP datasets to understand the extent and impact of class imbalance.
- Develop strategies to manage and mitigate class imbalance.
- Ensure a fair and balanced representation of defects and non-defects classes within training data.

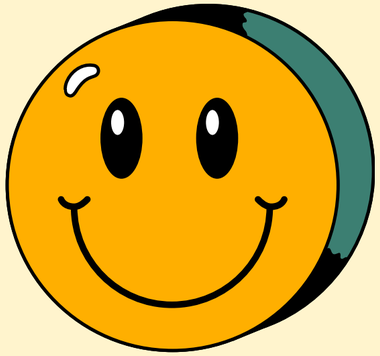
Computational Complexity

- Identify and address computational challenges in traditional ML models for SDP.
- Examine the complexity and time requirements of these models.
- Propose and evaluate methods to reduce computational complexity and time consumption.
- Enable faster and more efficient processing of SDP tasks.

Significance of the Research

- The findings of this research will grant the field of swe a more effective approach to SDP.
- The hybrid ML model can help software developers and testers
  - identify defects earlier in the development process
  - reducing costs
  - improving software quality
- The techniques employed to address class imbalance, and computational complexity can be applied to other areas of ML research.

Literature Review



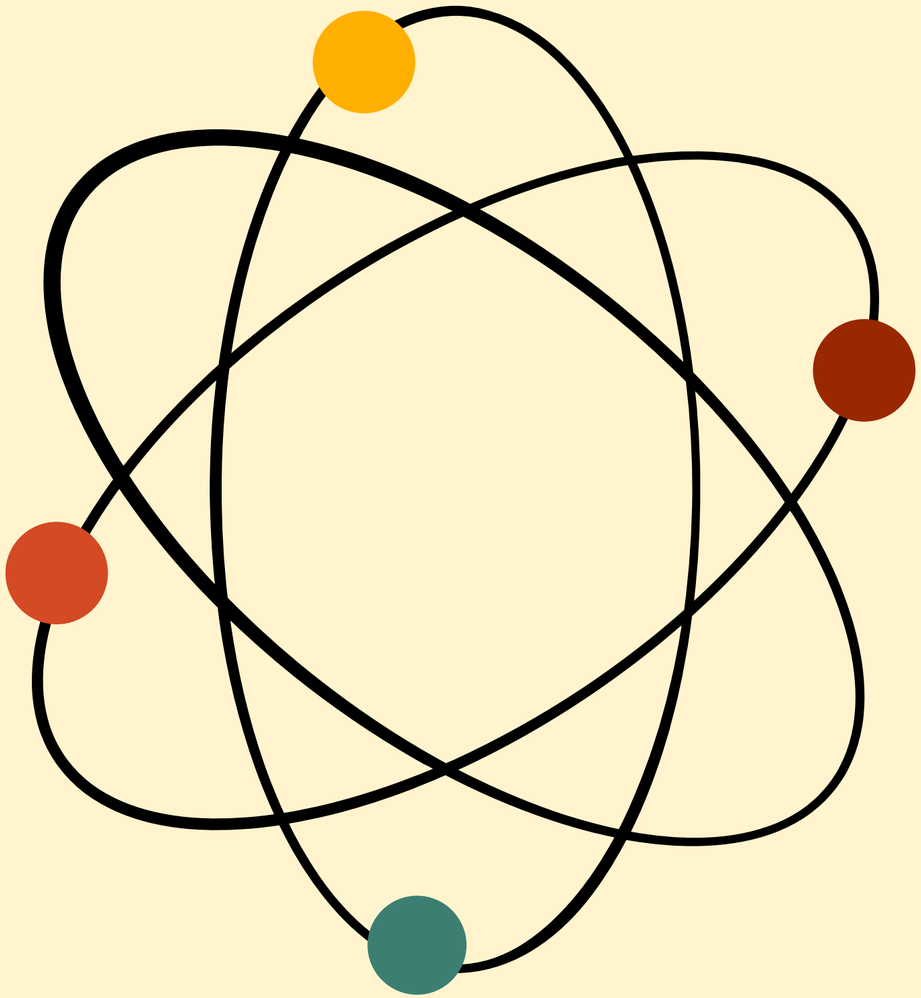
What is SDP?

- **What is Software Defect Prediction (SDP)?**
  - using ML and statistical techniques to predict defects in software modules before they are deployed
  - analyze historical data, such as past defect logs, code metrics, and development process information
  - SDP models identify patterns that are indicative of potential defects in future software releases
- **Why is SDP important in software engineering?**
  - Improves Software Quality
  - Reduces Costs
  - Enhances Efficiency
  - Minimizes Risk

Class Imbalance in Dataset

When defective software instances are vastly outnumbered by non-defective ones, which can cause prediction models to prioritize the majority class and overlooking the minority class.

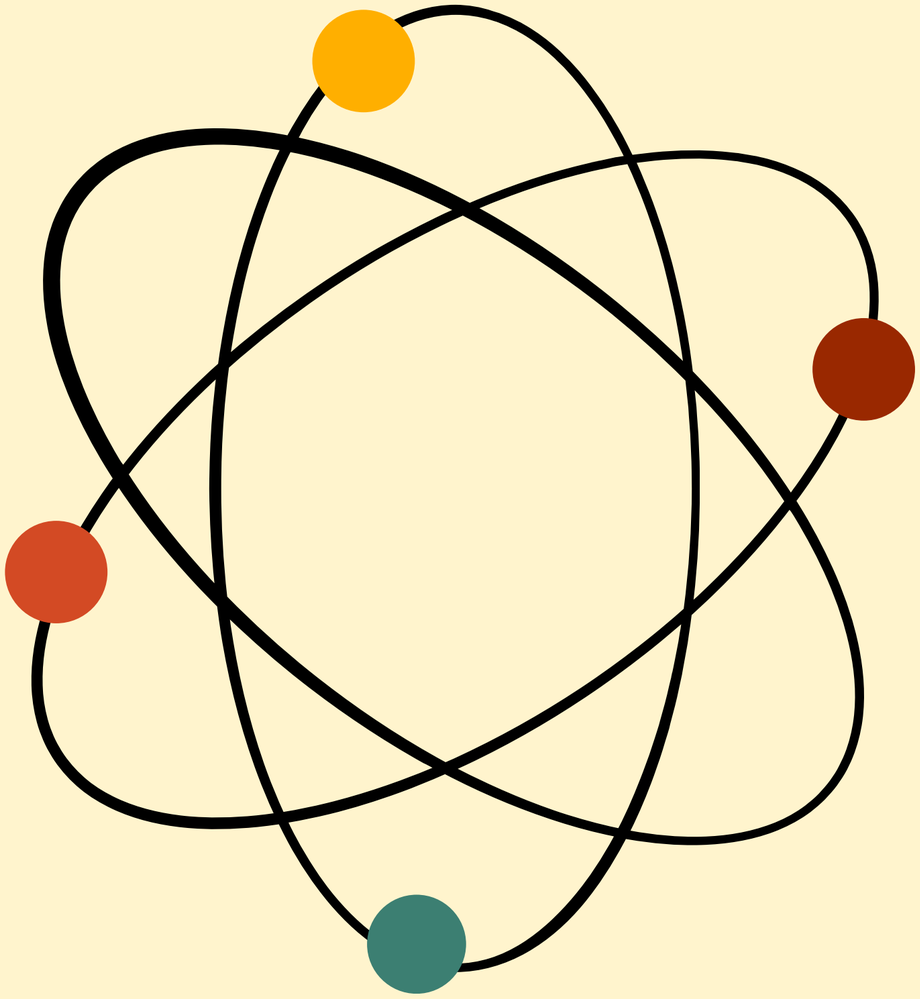
Hence, causing inaccuracy and inefficiency of the model used in SDP.



No	Techniques	Tools	Language	Dataset	Performance Metrics	Testing	Result
1	bGWO for feature selection. ML classifiers: KNN, DT, SVM, NB, ANN	Matlab (common)	Python	NASA (CM1, JM1, KC1, KC2, PC1) include metrics like McCabe's Cyclomatic Complexity, Halstead metrics, and line counts.	Accuracy, precision, recall, and F1-score.	Feature selection was performed using the bGWO to reduce dimensionality. ML classifiers (KNN, DT, SVM) were trained and tested using the selected features. Performance metrics such as accuracy, precision, recall, and F1-score were calculated on the test sets of the NASA datasets to evaluate the defect prediction models.	Improved prediction performance with optimal feature subsets.
2	Oversampling techniques (e.g., COSTE and SMOTE) to balance datasets. ML model: DT, RF, NB, NN	Jupyter Notebooks or IDEs like PyCharm (common)	Python (+libraries like Scikit-learn and TensorFlow)	NASA, PROMISE Github, Kaggle and other public sources for additional datasets	Accuracy, recall, precision, and ROC curves.	Testing involved addressing class imbalance with oversampling techniques (e.g., SMOTE and COSTE). ML models like RF, DT, and NN were evaluated. Metrics like precision, recall, accuracy, and ROC curves were used to validate the effectiveness of the defect prediction models.	Improved class distribution and defect prediction accuracy.
3	M-SMOTE for balancing data.	Tensorflow, pytorch	Python	Open-source datasets	Accuracy: 95.32%, Recall:	Data pre-processing included normalization and balancing with M-	Reduced overfitting and computational cost with

Result

- **SMOTE (Synthetic Minority Over-Sampling Technique)**
- Creates synthetic samples for minority class
  - Akhilesh G.V Dhavileswarapu et al. (2021) – SMOTE-Tuned with Neural Networks improved defect prediction accuracy.
  - Asmaa M Ibrahim et al. (2023) – Used SMOTE with Balanced Random Forest, achieving higher recall & AUC.
- **COSTE (Complexity-based Oversampling Technique)**
- Generates synthetic defective instances based on complexity similarity
  - Shuo Feng et al. (2021) – COSTE outperformed SMOTE in handling high-dimensional datasets with fewer false positives.
- **Ensemble Learning: Bagging, Boosting, Stacking**
  - Combines multiple weak learners to improve stability
  - Tarunim Sharma et al. (2023) – Boosting achieved 99% AUC in time-sensitive defect prediction scenarios.
  - Muhammad Jumare et al. (2024) – RF with SMOTE-Tomek achieved 82.3% accuracy and 96.8% recall.



Computational Complexity

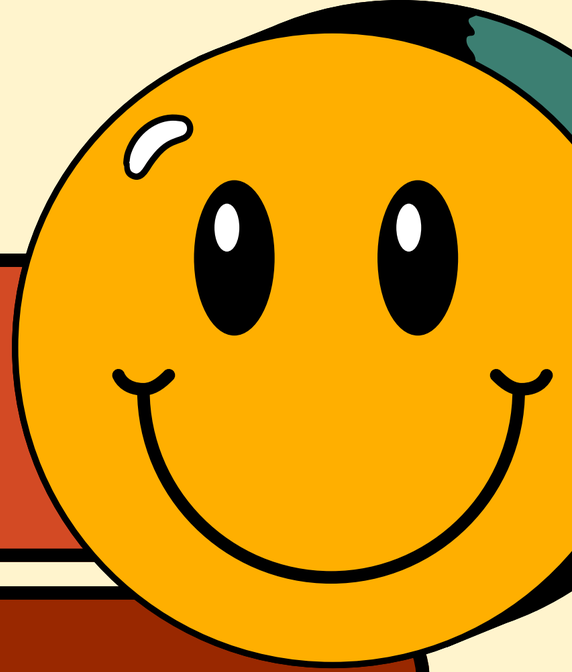
- Computational complexity studies the time and memory an algorithm needs to solve a problem.
- It helps determine an algorithm's efficiency with large datasets or complex tasks.
- By analyzing computational complexity, researchers can compare and choose the best algorithms for specific situations, optimizing performance in areas like machine learning and software development.
- This is essential for fast processing and efficient resource use.

No	Techniques	Tools	Language	Dataset	Performance Metrics	Testing	Result
1	M-SMOTE, F-BERT for feature extraction, Bi-LSTM with Convolutional GRU, HLR optimization.	Not specified, but DL frameworks are implied.	Not specified.	Open-source CI/CD datasets with labeled numerical data.	Accuracy (95.32%), Recall (93.3%), F1-score (91.355%), MCC (94.98%).	Dataset balancing, feature extraction, and classification were tested using cross-validation and time-domain labeling.	Reduced computational complexity and waiting time, high accuracy, and efficient defect prediction.
2	Bagging, Boosting, Stacking; compared with KNN, SVM, CNN, etc.	Not specified.	Implemented in a simulated environment using Python or similar tools for ML.	NASA and PROMISE repositories (Ant-1.7, PC1, JM1).	Accuracy, Precision, F1-score (0.97 average), G-Mean (0.96), AUC (0.99).	K-fold cross-validation (K=10), runtime comparison.	Boosting methods were efficient in time-sensitive scenarios; ensemble methods outperformed individual algorithms.
3	Ensemble methods: Bagging, Boosting, Stacking, and RF. Feature selection techniques to improve efficiency.	Not specified.	Not specified.	Various datasets from the PROMISE repository.	Accuracy, Precision, Recall, F1-score.	Evaluated computational efficiency and time consumption of ensemble methods.	Ensemble methods generally required more computational resources, with training times ranging from 15 to 30 minutes depending on

Result

- **Feature Selection Techniques (PSO, Genetic Algorithms)**
- Optimizes feature subsets for ML models
  - Yazan Al-Smadi et al. (2023) – Used PSO for feature selection, achieving 90%+ accuracy with gradient boosting.
  - Aimen Khalid et al. (2023) – K-Means clustering + PSO optimized SVM to 99.8% accuracy.
- **Hybrid Models & Optimized Ensemble Learning**
- Combining Models: Hybrid approaches leverage different ML strengths
  - RF for feature selection
  - SVM for classification
  - Misbah Ali et al. (2023) – RF, SVM, ANN ensemble improved accuracy beyond 20 prior models.
  - Xin Fan et al. (2024) – Stable Learning with ResNet improved cross-project defect prediction by 5.89–25.46%.

Chosen Method



Random Forest:

- excels in accuracy with large datasets and numerous features
- insight into feature importance aids in feature selection and dimensionality reduction

Support Vector Machine:

- performs well in high-dimensional spaces, identifying optimal hyperplanes for class differentiation
- effectively handles sparse data and enhances model performance through feature selection

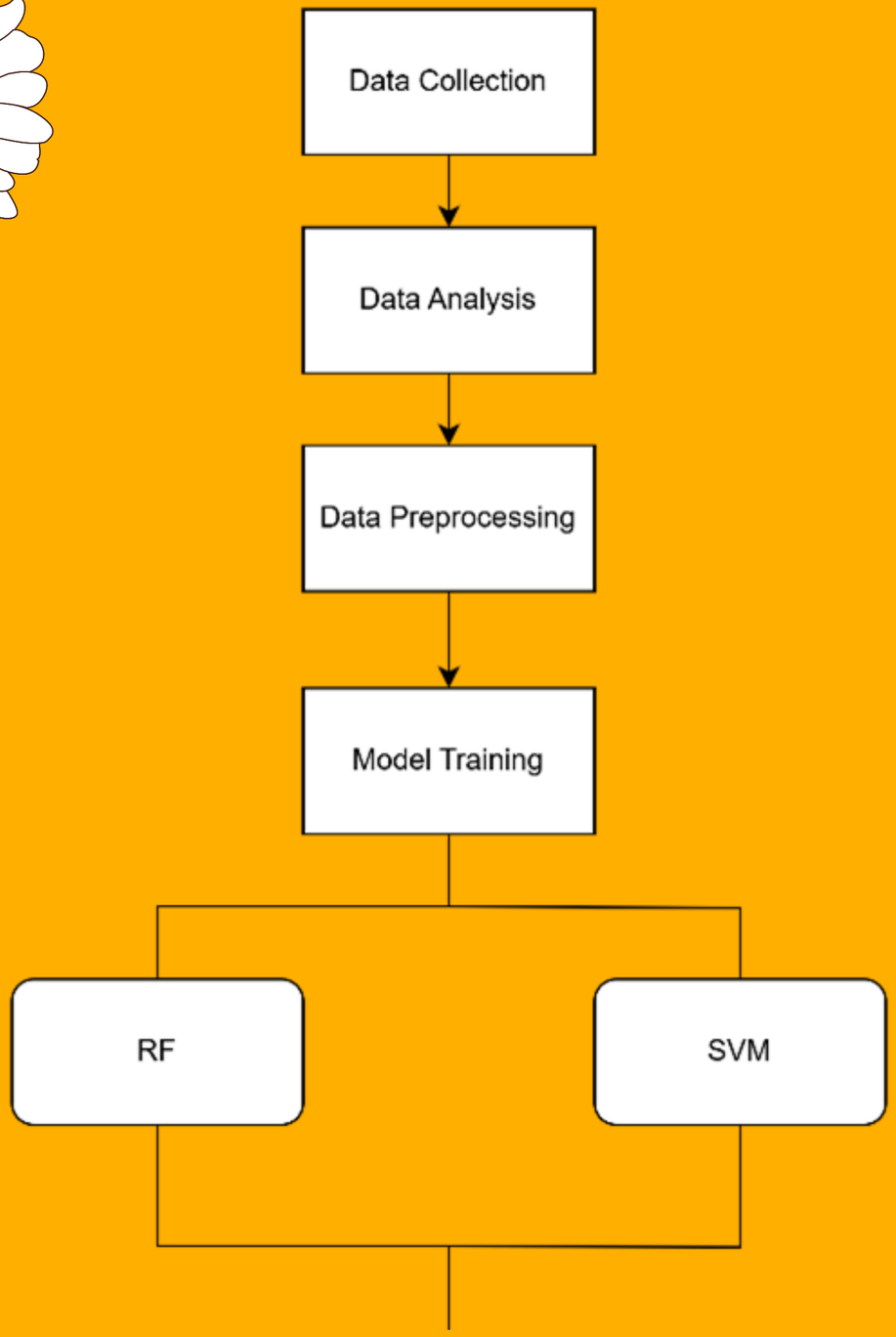
Hybrid of Both:

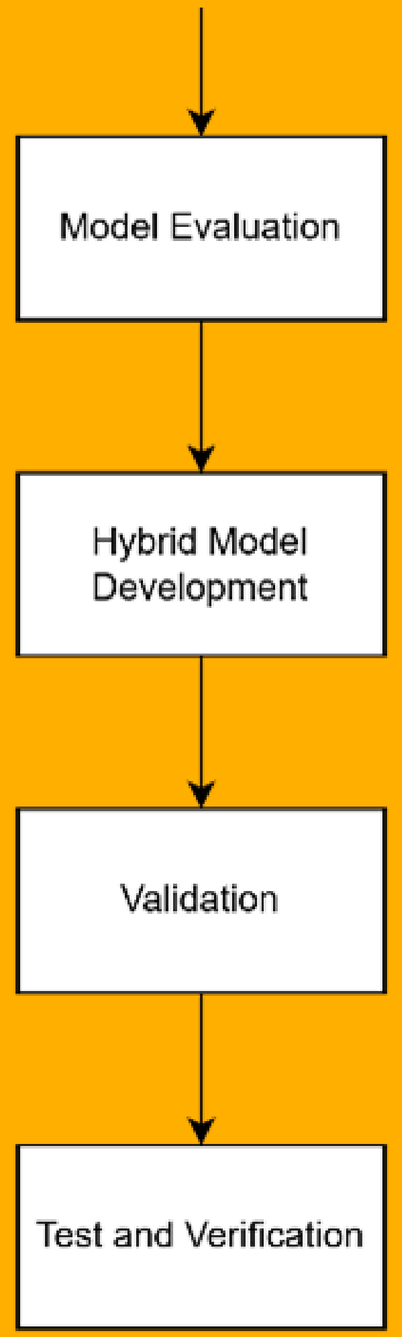
- offers comprehensive and accurate defect prediction, supported by empirical evidence showing superior performance metrics
- enhances software quality by identifying defect-prone modules efficiently

Research Gaps

- While significant progress has been made, more comprehensive studies are needed to combine these aspects into hybrid models.
- The proposed hybrid ML model aims to fill these gaps by combining effective feature selection techniques with robust RF and SVM methods
- Ultimately enhancing predictive accuracy and interpretability in SDP

Research Methodology


Data Collection
Data Analysis
Data Preprocessing
Model Training
RF
SVM


Model Evaluation
Hybrid Model Development
Validation
Test and Verification

Methodology

Data Collection

- Gathering and preparing numerical data
  - that contain instances of software defects
  - labeling them based on time domain limitations
- To ensuring that the data is comprehensive and representative of the various types of defects that may occur in software development.

Data Analysis

- Charts and graphs to visually represent the collected data
- Generating summary statistics to understand the data's distribution and characteristics
- To find patterns, trends, and anomalies inside the data

Data Preprocessing

- Cleaning and normalizing the data
- Removing any irrelevant or redundant information
- Handling missing values
- Adjusting the data to a consistent range
- Feature selection techniques to pinpoint the most pertinent features for the prediction model

Methodology

Model Training

- The training of both the RF and SVM models
- Learn from existing data and identify patterns that indicate the presence of software defects.
- Optimizing the models' parameters to maximize their predictive capabilities.

Model Evaluation

- Assessed by utilizing many performance metrics
  - accuracy, precision, recall, F1-score, and AUC-ROC.
- See how well each model predicts software faults.
- A comparative analysis of to determine their respective strengths and weaknesses.

Hybrid Model Development

- Combining the strengths of both the RF and SVM models to create an accurate prediction model.
- Aims to improve the entire predictive capability and address limitations of individual models.

Methodology

Validation

- Validation of the hybrid model is carried out on separate test datasets to ensure its generalizability and reliability.
- Test the model's capabilities on previously unread data to confirm its effectiveness in correctly envisioning software defects

Test and Verification

- Testing and verifying the hybrid model using log files and real-world data.
- Gives evidence of the model's effectiveness in practical applications.
- The model's performance is evaluated in real-world conditions, ensuring its reliability and robustness for SDP in continuous integration and continuous delivery (CI/CD) environments.

Data Collection

The image displays the command-line output of the initial rows of a dataset.
By checking these rows, the researcher can confirm the dataset's correct format and readiness for further analysis.
This ensures the data is accurately loaded, which is a crucial step before diving into deeper examination and preprocessing for the research.

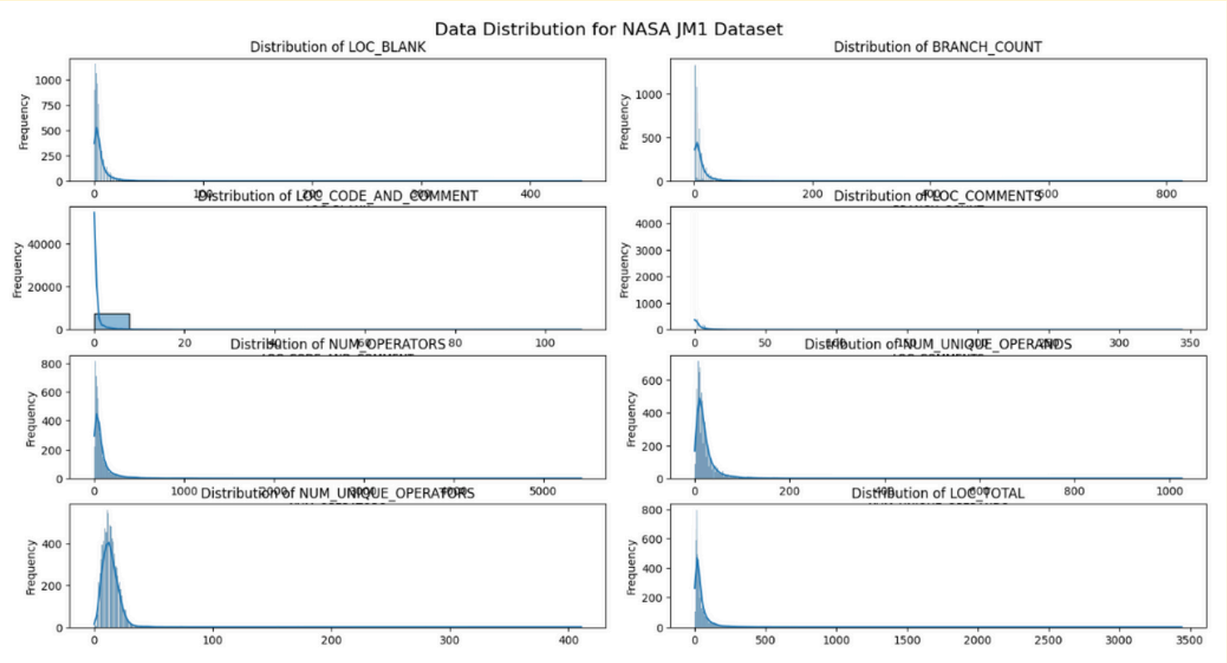
```
C:\Users\User\Downloads\268514>C:\Users\User\AppData\Local\Programs\Python\Python313\python.exe jm1.py
   LOC_BLANK  BRANCH_COUNT  LOC_CODE_AND_COMMENT  ...  NUM_UNIQUE_OPERATORS  LOC_TOTAL  label
0        1.0           7.0                   0.0  ...                  12.0       14.0   b'N'
1        5.0          37.0                   0.0  ...                  22.0       98.0   b'N'
2        2.0           1.0                   0.0  ...                   5.0       14.0   b'Y'
3       16.0           1.0                   0.0  ...                  12.0       70.0   b'Y'
4        0.0           7.0                   0.0  ...                  12.0       12.0   b'N'

[5 rows x 22 columns]
```

- Help **identify the central tendency** (mean, median) and variability (standard deviation, range) of the data, giving insights into the distribution.
- **Detecting Anomalies:** Examining the minimum, maximum, and quartile values.
- **Comparing Variables**: By summarizing different variables, you can easily compare their properties and distributions.

Data Analysis (Summary Statistic)

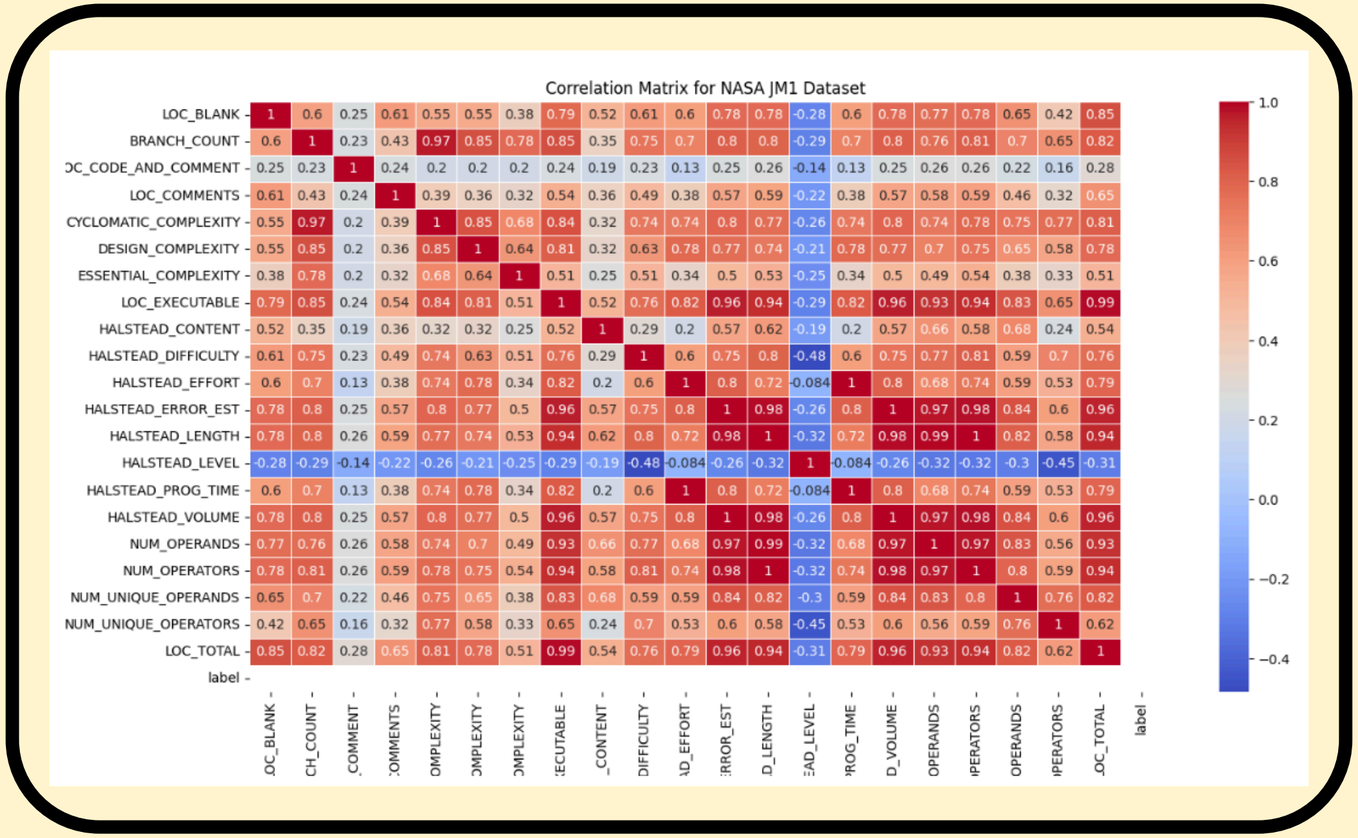
	LOC_BLANK	BRANCH_COUNT	LOC_CODE_AND_COMMENT	LOC_COMMENTS	...	NUM_OPERATORS	NUM_UNIQUE_OPERANDS	NUM_UNIQUE_OPERATORS	LOC_TOTAL
count	7782.000000	7782.000000	7782.000000	7782.000000	...	7782.000000	7782.000000	7782.000000	7782.000000
mean	6.036494	12.073246	0.493832	3.684143	...	87.588795	21.394500	13.814572	46.250835
std	10.981517	23.933234	2.222237	10.373401	...	165.444355	28.931756	10.046540	80.575204
min	0.000000	1.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000	1.000000
25%	1.000000	3.000000	0.000000	0.000000	...	21.000000	8.000000	9.000000	15.000000
50%	3.000000	7.000000	0.000000	0.000000	...	44.000000	15.000000	13.000000	26.000000
75%	7.000000	13.000000	0.000000	3.000000	...	90.000000	25.000000	17.000000	51.000000
max	447.000000	826.000000	108.000000	344.000000	...	5420.000000	1026.000000	411.000000	3442.000000

[8 rows x 21 columns]



Data Analysis (Data Distribution)

To further understand the data distribution, histograms with kernel density estimates (KDE) are plotted for each feature. These plots demonstrate a graphical depiction of the distribution of each metric, highlighting the frequency and spread of the data points. This helps in identifying any skewness, outliers, or patterns in the data that may be relevant for further analysis. By doing so, the author can gain a profound insight of the dataset and prepare it for subsequent modeling and analysis tasks.



- 1 signifies a perfect positive correlation.
- 0 signifies no correlation.
- -1 signifies a perfect negative correlation.

The heatmap visualizes these correlation coefficients, where the color intensity signifies the strength of the correlation.

- Strong positive correlations = bright red
- strong negative correlations = bright blue
- weak correlations = lighter colors

Data Analysis (Correlation Matrix)

By examining the correlation matrix, it is easy to identify which metrics have strong linear relationships with each other. This information is useful for feature selection, multicollinearity analysis, and understanding the dataset's underlying structure.

Data Preprocessing

Data Cleaning

Handling missing values is an important step in data preprocessing, as it makes sures that the dataset is complete and ready for investigation. Missing values can distort statistical analyses and models if not properly managed.

LOC_BLANK	0
BRANCH_COUNT	0
LOC_CODE_AND_COMMENT	0
LOC_COMMENTS	0
CYCLOMATIC_COMPLEXITY	0
LOC_CODE_AND_COMMENT	0
LOC_COMMENTS	0
CYCLOMATIC_COMPLEXITY	0
LOC_COMMENTS	0
CYCLOMATIC_COMPLEXITY	0
DESIGN_COMPLEXITY	0
ESSENTIAL_COMPLEXITY	0
LOC_EXECUTABLE	0
DESIGN_COMPLEXITY	0
ESSENTIAL_COMPLEXITY	0
LOC_EXECUTABLE	0
LOC_EXECUTABLE	0
HALSTEAD_CONTENT	0
HALSTEAD_DIFFICULTY	0
HALSTEAD_CONTENT	0

HALSTEAD_EFFORT	0
HALSTEAD_ERROR_EST	0
HALSTEAD_EFFORT	0
HALSTEAD_ERROR_EST	0
HALSTEAD_LENGTH	0
HALSTEAD_ERROR_EST	0
HALSTEAD_LENGTH	0
HALSTEAD_LENGTH	0
HALSTEAD_LEVEL	0
HALSTEAD_PROG_TIME	0
HALSTEAD_VOLUME	0
NUM_OPERANDS	0
NUM_OPERATORS	0
NUM_UNIQUE_OPERANDS	0
NUM_UNIQUE_OPERATORS	0
LOC_TOTAL	0

Data Preprocessing

Data Cleaning (ii)

Removing duplicates is an essential part of data cleaning, as duplicate rows can introduce bias and inaccuracies in the analysis.

```
Total number of duplicate rows: 0
Total number of duplicate rows after removal: 0
```

Data Preprocessing

Data Transformation

Normalization ensures data falls within a particular range, usually between 0 and 1. This process helps to bring all features to a similar scale without altering or skewing the variations in the value ranges.

```
Columns in the DataFrame:
Index(['LOC_BLANK', 'BRANCH_COUNT', 'LOC_CODE_AND_COMMENT', 'LOC_COMMENTS',
       'CYCLOMATIC_COMPLEXITY', 'DESIGN_COMPLEXITY', 'ESSENTIAL_COMPLEXITY',
       'LOC_EXECUTABLE', 'HALSTEAD_CONTENT', 'HALSTEAD_DIFFICULTY',
       'HALSTEAD_EFFORT', 'HALSTEAD_ERROR_EST', 'HALSTEAD_LENGTH',
       'HALSTEAD_LEVEL', 'HALSTEAD_PROG_TIME', 'HALSTEAD_VOLUME',
       'NUM_OPERANDS', 'NUM_OPERATORS', 'NUM_UNIQUE_OPERANDS',
       'NUM_UNIQUE_OPERATORS', 'LOC_TOTAL', 'label'],
      dtype='object')
Total number of duplicate rows: 0
Total number of duplicate rows after removal: 0
Total number of missing values in the label column: 7782
Standardized Data:
   LOC_BLANK  BRANCH_COUNT  LOC_CODE_AND_COMMENT  LOC_COMMENTS  CYCLOMATIC_COMPLEXITY  ...  NUM_OPERATORS  NUM_UNIQUE_OPERANDS  NUM_UNIQUE_OPERATORS  LOC_TOTAL  label
0  -0.458663     -0.211989             -0.222237     -0.355176              -0.197623  ...      -0.342063            -0.221034             -0.180628  -0.400283    0.0
1  -0.094391      1.041579             -0.222237      0.223264               0.867947  ...       0.703673             1.299884              0.814803   0.642288    0.0
2  -0.367595     -0.462702             -0.222237     -0.355176              -0.410737  ...      -0.408555            -0.221034             -0.877430  -0.400283    0.0
3   0.907356     -0.462702             -0.222237     -0.355176              -0.410737  ...       0.993819             1.680113             -0.180628   0.294764    0.0
4  -0.549731     -0.211989             -0.222237     -0.355176              -0.197623  ...      -0.426689            -0.497565             -0.180628  -0.425106    0.0

[5 rows x 22 columns]

Normalized Data:
   LOC_BLANK  BRANCH_COUNT  LOC_CODE_AND_COMMENT  LOC_COMMENTS  CYCLOMATIC_COMPLEXITY  ...  NUM_OPERATORS  NUM_UNIQUE_OPERANDS  NUM_UNIQUE_OPERATORS  LOC_TOTAL  label
0   0.002237      0.007273                   0.0      0.000000               0.006397  ...       0.005720             0.014620              0.029197   0.003778    0.0
1   0.011186      0.043636                   0.0      0.017442               0.038380  ...       0.037638             0.057505              0.053528   0.028189    0.0
2   0.004474      0.000000                   0.0      0.000000               0.000000  ...       0.003690             0.014620              0.012165   0.003778    0.0
3   0.035794      0.000000                   0.0      0.000000               0.000000  ...       0.046494             0.068226              0.029197   0.020052    0.0
4   0.000000      0.007273                   0.0      0.000000               0.006397  ...       0.003137             0.006823              0.029197   0.003197    0.0

[5 rows x 22 columns]
```

Data Preprocessing

Feature Engineering

Feature selection focuses on finding the most remarkable variables that contribute to the target variable. The complexity of the model will significantly lessen by picking only the most relevant features, leading to improved performance and reducing the risk of overfitting.

```
Columns in the DataFrame:
Index(['LOC_BLANK', 'BRANCH_COUNT', 'LOC_CODE_AND_COMMENT', 'LOC_COMMENTS',
       'CYCLOMATIC_COMPLEXITY', 'DESIGN_COMPLEXITY', 'ESSENTIAL_COMPLEXITY',
       'LOC_EXECUTABLE', 'HALSTEAD_CONTENT', 'HALSTEAD_DIFFICULTY',
       'HALSTEAD_EFFORT', 'HALSTEAD_ERROR_EST', 'HALSTEAD_LENGTH',
       'HALSTEAD_LEVEL', 'HALSTEAD_PROG_TIME', 'HALSTEAD_VOLUME',
       'NUM_OPERANDS', 'NUM_OPERATORS', 'NUM_UNIQUE_OPERANDS',
       'NUM_UNIQUE_OPERATORS', 'LOC_TOTAL', 'label'],
      dtype='object')
Total number of duplicate rows: 0
Total number of duplicate rows after removal: 0
Total number of missing values in the label column: 7782
C:\Users\User\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\feature_selection\_univariate_selection.py:107: RuntimeWarning: invalid value encountered in divide
  msb = ssbn / float(dfbn)
Selected Features:
Index(['HALSTEAD_ERROR_EST', 'HALSTEAD_LENGTH', 'HALSTEAD_LEVEL',
       'HALSTEAD_PROG_TIME', 'HALSTEAD_VOLUME', 'NUM_OPERANDS',
       'NUM_OPERATORS', 'NUM_UNIQUE_OPERANDS', 'NUM_UNIQUE_OPERATORS',
       'LOC_TOTAL'],
      dtype='object')
Dataset with Selected Features:
   HALSTEAD_ERROR_EST  HALSTEAD_LENGTH  HALSTEAD_LEVEL  HALSTEAD_PROG_TIME  HALSTEAD_VOLUME  ...  NUM_OPERATORS  NUM_UNIQUE_OPERANDS  NUM_UNIQUE_OPERATORS  LOC_TOTAL  label
0                0.09            59.0            0.09              174.56           280.54  ...           31.0                 15.0                  12.0       14.0    0.0
1                0.74           351.0            0.04             3388.22          2225.29  ...          204.0                 59.0                  22.0       98.0    0.0
2                0.05            37.0            0.35               25.17           159.91  ...           20.0                 15.0                   5.0       14.0    0.0
3                0.95           450.0            0.06             2697.42          2860.90  ...          252.0                 70.0                  12.0       70.0    0.0
4                0.04            26.0            0.13               47.33           110.45  ...           17.0                  7.0                  12.0       12.0    0.0

[5 rows x 11 columns]
```

Data Analysis Plan

FYP1 GANTT CHART

No	Task	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14
1.0	**FYP 1 Report**														
	Chapter 1: Introduction														
	Chapter 2: Literature Review														
	Chapter 3: Research Methodology														
	Chapter 4: Data Analysis Plan														
	Chapter 5: Conclusion														
	Meeting Logs														
	Plagiarism Check														
2.0	**Research Methodology**														
	Data Collection														
	Data Analysis														
	Data Preprocessing														
3.0	**FYP 1 Oral Presentation**														
	Presentation Slide														
	Presentation Practice														

FYP2 GANTT CHART

No	Task	W1	W 2	W3	W 4	W5	W 6	W7	W 8	W 9	W10	W1 1	W12	W13	W14	W1 5	W16
1.0	**FYP 1 Report**																
	Chapter 1: Introduction																
	Chapter 2: Literature Review																
	Chapter 3: Theoretical Framework																
	Chapter 4: Research Framework																
	Chapter 5: Data Analysis and Results																
	Chapter 6: Discussions																
	Chapter 7: Conclusion and Recommendations																
	Meeting Logs																
	Plagiarism Check																
2.0	**Prototype**																
	Data Collection																
	Data Analysis																
	Data Preprocessing																
	Model Training																
	Model Evaluation																

FYP2 GANTT CHART

No	Task	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	W16
	Hybrid Model Development																
	Validation																
	Test and Verification																
3.0	**FYP 1 Oral Presentation**																
	Presentation Slide																
	Presentation Practice																
	Presentation Poster																

Conclusion

- This study addressed major challenges in SDP by proposing a hybrid RF-SVM model.
- The literature review revealed gaps in handling class imbalance and computational efficiency, leading to the development of an optimized prediction framework.
- The research methodology outlined data collection, analysis and preprocessing strategies. In the next phase, model training and hybrid model development, validation and testing will be conducted to evaluate the model's effectiveness.
- The anticipated outcomes will contribute to the field by improving defect detection accuracy and reducing computational overhead.

Thank You